

The Complete Odoo Coderbyte Assessment Masterclass

A Zero-to-Expert Preparation Guide for Python Developers

String Manipulation | SQL & Database Design | Graph Algorithms (BFS) | Stacks & Queues | Arrays & Two Pointers | Math & Logic

June 2026 Edition

Chapter 1: How to Use This Guide

This guide is designed so that **you never have to look anything up**. Each chapter follows this structure: The Concept (what is this thing, mathematically and intuitively?), The Derivation (why does the algorithm work?), The Pattern (when do you recognize this problem type?), The Code (clean, typed Python), The Pitfalls (what breaks this?), and The Practice (problems with hints, then solutions). Study method: Read the concept first. Cover the code. Try to write it yourself. Check. Repeat.

Chapter 2: The Assessment

The Odoo Coderbyte assessment is a **general coding proficiency test** — not an Odoo framework quiz. Platform: Coderbyte (browser-based, webcam proctored, session recorded). Duration: 1 hour. Languages: Python and Java. Question count: Typically 2-3 coding problems. Difficulty: ~67% Easy, ~33% Medium. Typical mix: 1 String/DSA + 1 SQL/Database design + optionally 1 Graph/Stack problem.

Proctoring Reality

Coderbyte monitors `document.visibilitychange` and `window.onblur` to detect tab switching and window focus loss. It also records your webcam session. Human reviewers inspect flagged recordings. The next interview round requires you to **explain and optimize your solutions** — so you must fully understand every line you submit.

NOTE: The technical interview after this round will ask you to walk through your Coderbyte code. If you cannot explain your approach, you will be disqualified regardless of whether your code passed.

Pipeline Overview

1. Coderbyte Assessment (coding + proctoring). 2. Aptitude & English Test (MCQs + email writing). 3. Technical Interview (explain solutions, live coding, OOP). 4. HR / Final Round.

The Problem-Solving Framework

When you see a problem, do not code immediately. Use **R.A.P.I.D.**: R — Read carefully. Identify inputs, outputs, constraints. A — Analyze examples. Trace through small cases by hand. P — Pattern match. Does this feel like a Two-Pointer? Stack? Graph? I — Implement brute force first if stuck. A working slow solution is better than a broken fast one. D — Debug edge cases. Empty input, single element, all same elements, maximum constraints.

Time allocation for 1 hour: 0-5 min: Read all problems. Pick the easiest. 5-25 min: Solve Problem 1 completely. 25-45 min: Solve Problem 2. 45-55 min: Problem 3 or optimize/review. 55-60 min: Final sanity checks.

TIP: Never start with the hardest problem. Momentum matters. Solving an Easy problem in 15 minutes gives you confidence and time buffer for the Medium.

Chapter A: Data Structures Fundamentals

Before solving problems, you must understand the building blocks. This chapter covers algorithms, Big-O complexity, arrays, hash tables, stacks, queues, and graphs from first principles.

A.1 What is an Algorithm?

An **algorithm** is a finite sequence of well-defined instructions to solve a problem. In coding assessments, we care about three things: **Correctness** — does it produce the right output for all valid inputs? **Efficiency** — how much time and memory does it consume? **Clarity** — can another human understand it?

A.2 Time & Space Complexity (Big-O)

Big-O describes how resource usage grows as input size n grows. **$O(1)$** — Constant time. The operation takes the same time regardless of input size. Example: accessing an array by index. **$O(\log n)$** — Logarithmic. The problem size halves each step. Example: binary search. **$O(n)$** — Linear. You touch each element once. Example: finding the maximum in an array. **$O(n \log n)$** — Linearithmic. Common in efficient sorting. **$O(n^2)$** — Quadratic. Nested loops over the same data. **$O(2^n)$** — Exponential. Avoid for $n > 30$.

Space Complexity: $O(1)$ extra space means you only use a fixed number of variables, regardless of input size. Modifying the input array in-place is often considered $O(1)$ extra. $O(n)$ extra space means you allocate additional storage proportional to input (e.g., a hash map or a copy of the array).

NOTE: The limits rule: $n \leq 20$ means $O(2^n)$ might be acceptable. $n \leq 1,000$ means $O(n^2)$ is usually fine. $n \leq 100,000$ means you need $O(n \log n)$ or $O(n)$. $n \leq 10^6$ means $O(n)$ is expected.

A.3 Arrays, Lists, and Memory

An **array** is a contiguous block of memory where each element is the same size. This allows $O(1)$ access by index: the computer calculates $\text{address} = \text{base_address} + \text{index} * \text{element_size}$. In Python, list is a dynamic array: indexing is $O(1)$, append is $O(1)$ amortized, pop from end is $O(1)$, pop from front is $O(n)$ because elements must shift, search in unsorted is $O(n)$.

TIP: If you need to frequently remove from the front, use `collections.deque`, not a list.

A.4 Hash Tables (Dictionaries/Sets)

A **hash table** maps keys to values using a hash function. In Python, dict and set are hash tables. Operations: Insert $O(1)$ average, Lookup $O(1)$ average, Delete $O(1)$ average. The catch: worst case is $O(n)$ if all keys collide, but Python hash table implementation is highly optimized and this is practically impossible with random data. Use for: counting frequencies (Counter), checking existence (Set), mapping relationships (Dictionary), caching results (Memoization).

A.5 Stacks & Queues as Abstract Data Types

An **Abstract Data Type (ADT)** defines behavior, not implementation. **Stack (LIFO)** — Last In, First Out. Operations: push (add to top), pop (remove from top), peek (view top). Real-world analogy: stack of plates. Python: list with `append()` and `pop()`. **Queue (FIFO)** — First In, First Out. Operations: enqueue (add to back), dequeue (remove from front), peek (view front). Real-world analogy: line at a grocery store. Python: `collections.deque` with `append()` and `popleft()`. Never use `list.pop(0)` — it is $O(n)$.

A.6 Graphs: The Math Behind Connections

A **graph** $G = (V, E)$ consists of V (a set of vertices/nodes) and E (a set of edges/connections between nodes). **Undirected**: edge (u, v) means u connects to v and v connects to u . **Directed**: edge (u, v) means u points to v . **Weighted**: each edge has a numerical value. **Unweighted**: all edges are equal. Key properties: Degree (number of edges connected to a node), Path (sequence of edges from A to B), Cycle (path starting and ending at same node), Connected (every node reachable from every other). Why graphs matter: shortest route, dependency resolution, network analysis. Odoos assessments reportedly include unweighted shortest path problems.

Chapter B: String Manipulation

String manipulation is the most frequently reported topic on the Odoo assessment. This chapter covers strings as sequences, two-pointer techniques, parsing with state machines, run-length encoding, pattern matching, and HTML validation from first principles.

B.1 Strings as Sequences

In Python, a string is an **immutable sequence of Unicode characters**. Immutable means you cannot change a character in-place: `s[0] = "x"` raises a `TypeError`. Any "modification" creates a new string. Memory implication: concatenating strings in a loop with `+` creates $O(n^2)$ total characters because each `+` allocates a new string. Always build a list and `join()`.

Essential operations (memorize these):

```
s = "  Hello, World!  "

# Inspection
len(s)           # 17
s[0]             # ' ' (first char)
s[-1]            # ' ' (last char)
s[3:8]           # 'Hello' (slice: start inclusive, end exclusive)
s[::-1]          # ' !dlrow ,olleH ' (reverse)

# Cleaning
s.strip()         # 'Hello, World!' — remove outer whitespace
s.lstrip()        # 'Hello, World!  '
s.rstrip()        # '  Hello, World!'

# Case
s.lower()         # '  hello, world!  '
s.upper()         # '  HELLO, WORLD!  '
s.title()         # '  Hello, World!  '

# Search & Replace
s.find("World")   # 9 (index of first match, or -1)
s.count("l")      # 3 (occurrences of substring)
s.replace("l", "x") # '  Hexxo, Worxd!  '

# Split & Join
s.split(",")      # ['  Hello', ' World!  ']
"-".join(["a","b"]) # 'a-b'

# Checks (return True/False)
"abc".isalpha()   # True (all alphabetic)
"123".isdigit()   # True (all numeric)
"a1".isalnum()    # True (alpha or numeric)
"   ".isspace()   # True (all whitespace)
"hello".startswith("he") # True
```

TIP: In a timed test, you cannot afford to look up methods. Drill these until they are muscle memory.

B.2 The Two-Pointer Paradigm

The concept: Instead of using extra memory to create a new data structure, use two indices (pointers) to traverse the existing structure. Why it works: Many problems have symmetry or ordering that allows you to process from both ends or with a fast/slow runner.

Variant 1: Opposite Pointers

Place one pointer at the start, one at the end. Move them toward the center. When to use: Palindromes, reversing in-place, two-sum on sorted array.

```
def is_palindrome(s: str) -> bool:
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True

# Time: O(n), Space: O(1)
# Trace for "racecar":
# left=0('r'), right=6('r') -> match. left=1, right=5.
# left=1('a'), right=5('a') -> match. left=2, right=4.
# left=2('c'), right=4('c') -> match. left=3, right=3.
# left == right, loop ends. Return True.
```

Variant 2: Same-Direction Pointers (Fast & Slow)

Both start at the beginning. One moves faster or conditionally. When to use: Remove duplicates, move zeroes, find middle of linked list.

```
def remove_duplicates(nums: list[int]) -> int:
    if not nums:
        return 0
    write = 1 # where to write the next unique element
    for read in range(1, len(nums)):
        if nums[read] != nums[read - 1]:
            nums[write] = nums[read]
            write += 1
    return write

# Trace [1, 1, 2, 2, 3]:
# read=1, nums[1]=1 == nums[0]=1 -> skip.
# read=2, nums[2]=2 != nums[1]=1 -> nums[1]=2, write=2.
# read=3, nums[3]=2 == nums[2]=2 -> skip.
# read=4, nums[4]=3 != nums[3]=2 -> nums[2]=3, write=3.
# Return 3. First 3 elements are valid.
```

B.3 Parsing & State Machines

A **state machine** (or finite automaton) is a mathematical model of computation. It has: States (the current condition), Transitions (rules for moving between states based on input), Start state, and Accept states (valid ending conditions). Why use it for parsing: Strings often have grammar rules. A state machine enforces those rules step-by-step.

Example: Validate a simple ID format (2 letters + 4 digits). States: S0: Start (expecting letter). S1: Got 1 letter. S2: Got 2 letters (expecting digit). S3-S5: Got 2 letters + 1-3 digits. S6: Got 2 letters + 4 digits (accept).

```
def validate_id(code: str) -> bool:
    if len(code) != 6:
        return False
    return code[:2].isalpha() and code[2:].isdigit()

# This is a compact state machine. For complex grammars,
# you would write explicit state transitions.
```

B.4 Run-Length Encoding (RLE)

The concept: Lossless data compression where consecutive identical characters are stored as a single character and count. The algorithm: Initialize count = 1. Scan left to right. If current char == previous char, increment count. If different, append (previous_char + count) to result, reset count = 1. After the loop, append the final run.

NOTE: Why the final append is critical: The loop only emits when a change is detected. The last run never encounters a "change" inside the loop, so it must be emitted after. This is the most common RLE bug.

```
def run_length_encode(s: str) -> str:
    if not s:
        return ""
    result = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            result.append(s[i - 1] + str(count))
            count = 1
    result.append(s[-1] + str(count)) # CRITICAL
    return "".join(result)

# "aaabbc" -> "a3b2c1"
# Edge case "a" -> "a1" (loop doesn't run, final append handles it)
```

B.5 Pattern Matching & Wildcards

The concept: Determine if a text string matches a pattern containing special symbols. Simplified wildcard rules: "?" matches exactly one character (any character). "*" matches any sequence of characters (including empty). The greedy approach: When we encounter "*", we do not know how many characters it should match. Assume "*" matches zero characters. If we later hit a mismatch, backtrack: assume "*" matches one more character than before. Mathematically, this is equivalent to trying all possible matches for "*" but doing so efficiently by remembering where the last "*" was.

```
def is_match(s: str, p: str) -> bool:
    s_idx = p_idx = 0
    star_idx = s_tmp_idx = -1
    while s_idx < len(s):
        if p_idx < len(p) and p[p_idx] in {s[s_idx], '?'}:
            s_idx += 1
            p_idx += 1
        elif p_idx < len(p) and p[p_idx] == '*':
            star_idx = p_idx
            s_tmp_idx = s_idx
            p_idx += 1
        elif star_idx == -1:
            return False
        else:
            p_idx = star_idx + 1
            s_tmp_idx += 1
            s_idx = s_tmp_idx
    return all(c == '*' for c in p[p_idx:])

# Trace: s="abc", p="a*c"
# s=0('a'), p=0('a') -> match. s=1, p=1.
# s=1('b'), p=1('*') -> star. star=1, tmp=1, p=2.
# s=1('b'), p=2('c') -> mismatch. Backtrack: p=2, tmp=2, s=2.
# s=2('c'), p=2('c') -> match. s=3, p=3. Done. True.
```

B.6 HTML/XML Validation with Stacks

The concept: HTML/XML has a tree structure. Tags must be properly nested: is valid; is not. Why stacks: The most recently opened tag must be the next one closed. This is LIFO behavior. Algorithm: Scan for tags. Push opening tags. On closing tag, pop and check match. Stack must be empty at end.

```
def validate_html_tags(html: str) -> bool:
    stack = []
    i = 0
    while i < len(html):
        if html[i] == '<':
            j = html.find('>', i)
            if j == -1:
                return False # unclosed tag
            tag = html[i+1:j]
            if tag.startswith('/'):
                if not stack or stack[-1] != tag[1:]:
                    return False
                stack.pop()
            else:
                tag_name = tag.split()[0]
                stack.append(tag_name)
            i = j
        i += 1
    return len(stack) == 0
```

NOTE: In the actual Coderbyte variant, the logic may be more nuanced — e.g., determining if changing ONE element could fix the structure. Understand the core stack principle first, then adapt to the specific prompt.

B.7 Practice Problems

Problem 1: Valid Palindrome (Alphanumeric Only)

Given a string, determine if it is a palindrome considering only alphanumeric characters and ignoring cases. Example: "A man, a plan, a canal: Panama" returns True.

```
def is_palindrome(s: str) -> bool:
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

Time: O(n), Space: O(1)
Nested while loops skip non-alphanumeric characters.

Problem 2: String Compression (RLE with Length Check)

Perform basic string compression using counts of repeated characters. If the compressed string is not smaller than the original, return the original string.

```
def compress(s: str) -> str:
    if not s:
        return s
    compressed = []
    count = 1
    for i in range(1, len(s)):
```

```

        if s[i] == s[i-1]:
            count += 1
        else:
            compressed.append(s[i-1] + str(count))
            count = 1
        compressed.append(s[-1] + str(count))
    result = "".join(compressed)
    return result if len(result) < len(s) else s

# "aabcccccaaa" -> "a2b1c5a3"
# "abc" -> "abc" (compressed is longer)

```

Problem 3: First Non-Repeating Character

Find the first non-repeating character in a string and return its index. If it does not exist, return -1.

```

def first_uniq_char(s: str) -> int:
    from collections import Counter
    count = Counter(s)
    for i, ch in enumerate(s):
        if count[ch] == 1:
            return i
    return -1

# Time: O(n), Space: O(1) [at most 128 characters]

```

Chapter C: SQL & Database Design

The second most common question type on the Odoo assessment. You may be asked to write queries with JOIN, GROUP BY, and aggregation, or to design a schema for a simple system. This chapter covers relational algebra, schema design from first principles, normalization, query execution order, JOINS, and window functions.

C.1 What is Relational Algebra?

Relational databases are built on mathematical "relations" (tables). A relation is a set of tuples (rows) with named attributes (columns). Core operations: **Selection (σ)** — choose rows based on condition → SQL WHERE. **Projection (π)** — choose specific columns → SQL SELECT. **Union ($\cup$)** — combine two sets, removing duplicates → SQL UNION. **Difference ($-$)** — rows in one set but not another → SQL EXCEPT. **Cartesian Product ($\times$)** — all combinations → SQL CROSS JOIN. **Join ($\bowtie$)** — Cartesian product + selection → SQL JOIN. Understanding this helps you reason about what a query does, not just memorize syntax.

C.2 Schema Design from First Principles

The goal: Organize data to minimize redundancy, ensure integrity, and support queries efficiently. Step-by-step: 1. Identify entities (nouns in the problem). 2. Define attributes. 3. Identify relationships (1:N, N:M). 4. Assign primary keys. 5. Add foreign keys. 6. Add constraints (NOT NULL, UNIQUE, CHECK, DEFAULT). 7. Consider indexing.

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY AUTO_INCREMENT,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE courses (  
    course_id INT PRIMARY KEY AUTO_INCREMENT,  
    title VARCHAR(200) NOT NULL,  
    instructor_id INT,  
    credits INT CHECK (credits > 0),  
    FOREIGN KEY (instructor_id) REFERENCES instructors(instructor_id)  
);  
  
CREATE TABLE enrollments (  
    enrollment_id INT PRIMARY KEY AUTO_INCREMENT,  
    student_id INT NOT NULL,  
    course_id INT NOT NULL,  
    enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (student_id) REFERENCES students(student_id),  
    FOREIGN KEY (course_id) REFERENCES courses(course_id),  
    UNIQUE(student_id, course_id)  
);
```

TIP: In a design question, write CREATE TABLE statements. Even if the platform only asks for a description, providing SQL shows precision. Always mention foreign keys and junction tables.

C.3 Normalization (1NF, 2NF, 3NF)

1NF — All columns contain atomic (indivisible) values. No repeating groups. Violation: A courses column containing "Math, Physics" as a single string. Fix: Separate table. **2NF** — Must be in 1NF. No partial dependency: non-key attributes must depend on the entire primary key. Relevant only for composite keys. Violation: Enrollment table with PK(student_id, course_id) and a column student_name that

depends only on `student_id`. Fix: Move `student_name` to Student table. **3NF** — Must be in 2NF. No transitive dependency: non-key attributes must depend only on the primary key, not on other non-key attributes. Violation: A courses table with `instructor_name` (depends on `instructor_id`, not directly on `course_id`). Fix: Create an instructors table.

C.4 Query Execution Logic

SQL executes in this order (not top-to-bottom!): 1. FROM + JOINS (identify tables and combine). 2. WHERE (filter rows). 3. GROUP BY (group rows into buckets). 4. HAVING (filter groups). 5. SELECT (choose columns and compute expressions). 6. ORDER BY (sort). 7. LIMIT (truncate). Why this matters: You cannot use a column alias defined in SELECT inside a WHERE clause, because WHERE executes before SELECT.

```
-- WRONG: alias 'total' not available in WHERE
SELECT student_id, COUNT(*) AS total
FROM enrollments
WHERE total > 3      -- ERROR!
GROUP BY student_id;

-- CORRECT: use HAVING for group filters
SELECT student_id, COUNT(*) AS total
FROM enrollments
GROUP BY student_id
HAVING COUNT(*) > 3;
```

C.5 JOINS Deep Dive

INNER JOIN: Returns only rows where the join condition matches in both tables. **LEFT JOIN**: Returns all rows from the left table, and matching rows from the right. If no match, right-side columns are NULL. **RIGHT JOIN**: Opposite of LEFT. Rarely used because you can swap table order. **FULL OUTER JOIN**: Returns all rows from both tables, with NULLs where no match. MySQL does not support this natively; use UNION of LEFT and RIGHT. **SELF JOIN**: Join a table to itself. Useful for hierarchical data (employees and managers).

```
-- INNER JOIN example
SELECT s.name, c.title
FROM students s
INNER JOIN enrollments e ON s.student_id = e.student_id
INNER JOIN courses c ON e.course_id = c.course_id;

-- LEFT JOIN: find students with no enrollments
SELECT s.name
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
WHERE e.enrollment_id IS NULL;

-- SELF JOIN: employees and their managers
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

C.6 Aggregation & Window Functions

Aggregation functions: COUNT, SUM, AVG, MAX, MIN. They collapse groups into single values. **Window functions**: Perform calculations across a set of rows related to the current row, without collapsing them. RANK assigns rank with gaps for ties. DENSE_RANK is like RANK but no gaps. ROW_NUMBER is unique even for ties. PARTITION BY resets ranking per group.

```
-- RANK with gaps for ties
SELECT student_id, grade,
       RANK() OVER (ORDER BY grade DESC) AS rank
```

```
FROM scores;
```

```
-- PARTITION BY: top per course
```

```
SELECT course_id, student_id, grade,
```

```
       RANK() OVER (PARTITION BY course_id ORDER BY grade DESC) AS course_rank
```

```
FROM scores;
```

```
-- When to use window functions vs GROUP BY:
```

```
-- GROUP BY: collapse rows into summary statistics.
```

```
-- Window functions: keep all rows and add computed columns.
```

C.7 Practice Problems

Problem 1: Design an Animal Care Center Schema

Track animals, their owners, veterinarians, and appointments. An animal has one owner. A veterinarian sees many animals. An animal has many appointments.

```
CREATE TABLE owners (  
    owner_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    phone VARCHAR(20)  
);  
  
CREATE TABLE veterinarians (  
    vet_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    specialization VARCHAR(50)  
);  
  
CREATE TABLE animals (  
    animal_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    species VARCHAR(50),  
    owner_id INT,  
    FOREIGN KEY (owner_id) REFERENCES owners(owner_id)  
);  
  
CREATE TABLE appointments (  
    appointment_id INT PRIMARY KEY AUTO_INCREMENT,  
    animal_id INT NOT NULL,  
    vet_id INT NOT NULL,  
    appointment_date DATETIME NOT NULL,  
    notes TEXT,  
    FOREIGN KEY (animal_id) REFERENCES animals(animal_id),  
    FOREIGN KEY (vet_id) REFERENCES veterinarians(vet_id)  
);
```

Problem 2: Find Duplicate Emails

Write a SQL query to find all duplicate email addresses in a table named Person with columns id and email.

```
SELECT email  
FROM Person  
GROUP BY email  
HAVING COUNT(*) > 1;
```

Problem 3: Department Top 3 Salaries

The Employee table has id, name, salary, departmentId. The Department table has id, name. Find the top 3 unique salaries in each department.

```
SELECT d.name AS Department, e.name AS Employee, e.salary AS Salary  
FROM (  
    SELECT *,  
        DENSE_RANK() OVER (  
            PARTITION BY departmentId ORDER BY salary DESC  
        ) AS rnk  
    FROM Employee  
    ) e  
JOIN Department d ON e.departmentId = d.id  
WHERE e.rnk <= 3;
```

Chapter D: Graph Algorithms

Graph problems appear on Odoos assessments, typically as shortest path questions. Because edges are often unweighted (or have implicit weight of 1), Breadth-First Search (BFS) is the optimal algorithm. This chapter covers graphs from formal definitions through proofs to implementation.

D.1 What is a Graph? (Formal Definition)

A **graph** $G = (V, E)$ is a mathematical structure where V is a finite set of vertices (nodes) and E is a set of edges, where each edge is a pair (u, v) with $u, v \in V$. **Undirected**: $(u, v) \in E$ implies $(v, u) \in E$. The edge has no direction. **Directed**: (u, v) means there is a connection from u to v . **Weighted**: Each edge has an associated weight $w(e)$ in $\mathbb{R}$. **Unweighted**: All edges have implicit weight 1. Key terminology: Degree (number of edges incident to a vertex), Path (sequence of vertices where consecutive vertices are connected by edges), Path length (number of edges for unweighted, sum of weights for weighted), Shortest path (path with minimum length), Cycle (path starting and ending at same vertex), Connected graph (path between any two vertices).

D.2 Graph Representations

Adjacency Matrix: A 2D array $matrix[i][j]$ where $matrix[i][j] = 1$ if there is an edge from i to j , else 0. Pros: $O(1)$ edge lookup. Cons: $O(V^2)$ space. Wasteful for sparse graphs. **Adjacency List**: A dictionary where each vertex points to a list of neighbors. Pros: $O(V + E)$ space. Efficient for sparse graphs. Natural for traversal. Cons: Edge lookup is $O(\text{degree})$. For coding assessments, always use adjacency lists.

```
# Build adjacency list from edge list
edges = [[0, 1], [0, 2], [1, 3], [2, 3]]
graph = {}
for u, v in edges:
    graph.setdefault(u, []).append(v)
    graph.setdefault(v, []).append(u) # undirected

# Result: {0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2]}
```

TIP: Always use `setdefault()` or `collections.defaultdict(list)` to avoid `KeyError` when a node has no outgoing edges yet.

D.3 Breadth-First Search (BFS) — The Math & Proof

The concept: BFS explores a graph level by level. Starting from a source vertex, it visits all vertices at distance 1, then all at distance 2, then distance 3, etc. Algorithm: Initialize a queue with the source vertex and mark it visited. While the queue is not empty: dequeue a vertex u . For each neighbor v of u : if v is not visited, mark it visited, record its distance ($\text{distance}[u] + 1$), and enqueue it. Why a queue? A queue is FIFO. This ensures that vertices discovered earlier (closer to the source) are processed before vertices discovered later (farther away).

The Proof (Why BFS finds shortest paths in unweighted graphs):

Theorem: In an unweighted graph, BFS from source s computes the shortest path distance from s to every reachable vertex. Proof by induction on distance: Base case: The source s is at distance 0. BFS initializes with s in the queue. Correct. Inductive hypothesis: Assume BFS correctly computes distances for all vertices at distance less than or equal to k from s . Inductive step: Consider a vertex v at distance $k+1$ from s . By definition, there exists a vertex u at distance k such that (u, v) is an edge. By the inductive hypothesis, u is dequeued before any vertex at distance greater than k . When u is processed, v is discovered and assigned distance $k+1$. No shorter path to v can exist because any path to v must pass through a vertex at distance k , and we have already found the shortest paths to all such vertices. Conclusion: By induction, BFS computes correct shortest path distances for all vertices.

Time Complexity: $O(V + E)$. Each vertex is enqueued and dequeued at most once: $O(V)$. Each edge is examined exactly once (twice in undirected graphs): $O(E)$. Space Complexity: $O(V)$ for the visited set and queue.

```
from collections import deque

def bfs_shortest_path(graph, start, end):
    """
    graph: dict mapping node -> list of neighbors
    start: starting node
    end: target node
    Returns: shortest path length (number of edges), or -1 if unreachable.
    """
    if start == end:
        return 0

    visited = set([start])
    queue = deque([(start, 0)])

    while queue:
        node, dist = queue.popleft()
        for neighbor in graph.get(node, []):
            if neighbor == end:
                return dist + 1
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, dist + 1))

    return -1

# Trace: Graph {0: [1, 2], 1: [3], 2: [3], 3: []}
# Start=0, End=3
# Queue: [(0, 0)]. Visited: {0}.
# Dequeue (0, 0). Neighbors: 1, 2.
#   1 not visited -> enqueue (1, 1). Visited: {0, 1}.
#   2 not visited -> enqueue (2, 1). Visited: {0, 1, 2}.
# Queue: [(1, 1), (2, 1)].
# Dequeue (1, 1). Neighbors: 3.
#   3 == end! Return 1 + 1 = 2.
```

D.4 Why BFS and Not DFS for Shortest Path?

DFS (Depth-First Search) explores as deep as possible along one branch before backtracking. DFS might find a path $0 \rightarrow 1 \rightarrow 3$ (length 2) or $0 \rightarrow 2 \rightarrow 3$ (length 2) in a small example, but in a larger graph, it could find $0 \rightarrow 1 \rightarrow 4 \rightarrow 5 \rightarrow 3$ (length 4) before discovering the direct $0 \rightarrow 2 \rightarrow 3$ (length 2). DFS has no concept of "levels" — it just goes deep. When to use DFS: detecting cycles, topological sorting, finding connected components, problems where any path is sufficient. When to use BFS: shortest path in unweighted graphs, finding minimum number of steps, level-order traversal.

```
# Recursive DFS
def dfs_recursive(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    for neighbor in graph.get(start, []):
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited)
    return visited

# Iterative DFS (uses a stack instead of recursion)
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    while stack:
        node = stack.pop()
```

```

        if node not in visited:
            visited.add(node)
            for neighbor in reversed(graph.get(node, [])):
                if neighbor not in visited:
                    stack.append(neighbor)
    return visited

```

D.5 Common Graph Problem Patterns

Pattern 1: Shortest Path in a Grid

Many problems present a 2D grid where you can move in 4 or 8 directions. Each cell is a node; adjacent cells are edges.

```

from collections import deque

def shortest_path_binary_matrix(grid):
    """
    Find shortest clear path from top-left to bottom-right.
    0 = clear, 1 = blocked. Can move in 8 directions.
    """
    n = len(grid)
    if grid[0][0] == 1 or grid[n-1][n-1] == 1:
        return -1

    directions = [(-1,-1), (-1,0), (-1,1), (0,-1), (0,1), (1,-1), (1,0), (1,1)]
    queue = deque([(0, 0, 1)])
    grid[0][0] = 1 # mark visited by blocking

    while queue:
        r, c, dist = queue.popleft()
        if r == n-1 and c == n-1:
            return dist
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0:
                grid[nr][nc] = 1 # mark visited immediately
                queue.append((nr, nc, dist + 1))

    return -1

```

NOTE: Why mark visited immediately? If you wait until dequeuing, the same cell might be enqueued multiple times from different neighbors, wasting time and memory.

Pattern 2: Multi-Source BFS

When multiple starting points exist (e.g., multiple rotten oranges), enqueue all sources simultaneously.

```

from collections import deque

def oranges_rotting(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c, 0))
            elif grid[r][c] == 1:
                fresh += 1

    minutes = 0
    directions = [(0,1), (0,-1), (1,0), (-1,0)]

    while queue:

```

```
    r, c, minutes = queue.popleft()
    for dr, dc in directions:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
            grid[nr][nc] = 2
            fresh -= 1
            queue.append((nr, nc, minutes + 1))

    return minutes if fresh == 0 else -1
```

Chapter E: Stack & Queue Mastery

Stack problems are common on the Odo assessment, especially for validating nested structures and evaluating expressions. This chapter covers stacks and queues from abstract data type definitions through formal grammar proofs to advanced patterns like monotonic stacks.

E.1 The Stack Abstract Data Type

A **stack** is an ordered collection of items where addition and removal happen at the same end, called the **top**. Operations: `push(item)` — add to top, $O(1)$. `pop()` — remove and return top item, $O(1)$. `peek()/top()` — return top item without removing, $O(1)$. `is_empty()` — check if stack has items, $O(1)$. The LIFO principle: Last item pushed is the first item popped. This makes stacks ideal for: reversing things (input order is reversed on output), tracking nested structures (most recent opening must be closed first), undo mechanisms (most recent action is undone first), expression evaluation (postfix notation).

```
stack = []
stack.append(10)    # push
stack.append(20)
top = stack[-1]     # peek -> 20
stack.pop()         # pop -> 20
```

E.2 Parentheses & Tag Matching (Formal Grammar)

Balanced parentheses form a **context-free language**. The grammar is: $S \rightarrow (S) \mid [S] \mid \{S\} \mid SS \mid \epsilon$. This means a valid string is either empty, a pair of brackets containing another valid string, or two valid strings concatenated. Why a stack recognizes this grammar: Every opening bracket generates a "pending" requirement that must be satisfied by a matching closing bracket. The most recent pending requirement must be satisfied first — LIFO.

```
def is_valid_parentheses(s: str) -> bool:
    pair = {'(': ')', '[': ']', '{': '}'
    stack = []

    for ch in s:
        if ch in '([{':
            stack.append(ch)
        else:
            if not stack or stack[-1] != pair[ch]:
                return False
            stack.pop()

    return len(stack) == 0

# Time: O(n), Space: O(n)
# Edge cases:
# "(" -> False (stack not empty at end)
# ")" -> False (stack empty when encountering closing)
# "" -> True (empty stack at end)
```

TIP: Use a dictionary to map closing brackets to their opening counterparts. This is cleaner than multiple if-statements and easier to extend.

E.3 Evaluating Expressions (RPN & Infix)

Infix notation: Operators are between operands. $3 + 4 * 2$. Requires parentheses and precedence rules.

Postfix (Reverse Polish Notation): Operators follow operands. $3 4 2 * +$. No parentheses needed. The order is unambiguous. Why postfix is stack-friendly: When you see a number, you do not know what to do with it yet — save it. When you see an operator, you apply it to the two most recent numbers.

```
def eval_rpn(tokens: list[str]) -> int:
    stack = []
    ops = {'+', '-', '*', '/'}

    for token in tokens:
        if token not in ops:
            stack.append(int(token))
        else:
            b = stack.pop() # second operand
            a = stack.pop() # first operand
            if token == '+':
                stack.append(a + b)
            elif token == '-':
                stack.append(a - b)
            elif token == '*':
                stack.append(a * b)
            else:
                # Integer division truncating toward zero
                stack.append(int(a / b))

    return stack[0]

# Example: ["2", "1", "+", "3", "*"] -> ((2+1)*3) = 9
# Division trap: Python's // truncates toward negative infinity.
# Use int(a / b) to truncate toward zero.
```

NOTE: The division trap is critical. In Python, `-3 // 2` equals `-2`, but the problem expects `-1`. Always use `int(a / b)` for RPN problems, not `a // b`.

E.4 MinStack Design

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time. The trick is to store tuples of (value, current_min) so that each element carries the minimum of all elements below it.

```
class MinStack:
    def __init__(self):
        self.stack = [] # each element is (value, minsofar)

    def push(self, val: int) -> None:
        if not self.stack:
            self.stack.append((val, val))
        else:
            current_min = min(val, self.stack[-1][1])
            self.stack.append((val, current_min))

    def pop(self) -> None:
        self.stack.pop()

    def top(self) -> int:
        return self.stack[-1][0]

    def getMin(self) -> int:
        return self.stack[-1][1]

# All operations are O(1).
# Alternative: use two stacks (main stack + min stack).
```

E.5 Monotonic Stacks

A **monotonic stack** maintains elements in either increasing or decreasing order. When a new element violates the order, elements are popped until the order is restored. When to use: Finding the next greater element, previous greater element, or largest rectangle in histogram.

```
def daily_temperatures(temps: list[int]) -> list[int]:
    """
    For each day, find days until a warmer temperature.
    """
    n = len(temps)
    result = [0] * n
    stack = [] # stores indices with decreasing temperatures

    for i in range(n):
        while stack and temps[i] > temps[stack[-1]]:
            prev_idx = stack.pop()
            result[prev_idx] = i - prev_idx
            stack.append(i)

    return result

# Time: O(n), Space: O(n)
# How it works:
# The stack maintains indices where temperatures are in decreasing order.
# When we encounter a warmer temperature, it "resolves" all previous
# cooler days. Each index is pushed once and popped once.
```

E.6 Simplify Path (Unix-style)

Given a string path representing an absolute Unix path, simplify it. "." means parent directory, "." means current directory, multiple slashes should be treated as single slash.

```
def simplify_path(path: str) -> str:
    stack = []
    parts = path.split('/')

    for part in parts:
        if part == '' or part == '.':
            continue
        elif part == '..':
            if stack:
                stack.pop()
        else:
            stack.append(part)

    return '/' + '/'.join(stack)

# Example: "/home//foo/./bar/../baz" -> "/home/foo/baz"
```

E.7 Queue & BFS Connection

Why BFS uses a queue: BFS explores vertices in order of their distance from the source. A queue ensures FIFO behavior: vertices discovered earlier (closer to source) are processed before vertices discovered later (farther from source). Python queue implementation: collections.deque. Never use list.pop(0) for queues — it is O(n) because all elements must shift left. deque.popleft() is O(1).

```
from collections import deque

queue = deque()
queue.append(item) # enqueue (O(1))
item = queue.popleft() # dequeue (O(1))
front = queue[0] # peek (O(1))
```

Chapter F: Arrays & Two Pointers

Array problems on the Odoos assessment are typically Easy to Easy-Medium. The two-pointer technique is the most important pattern: it reduces time complexity from $O(n^2)$ to $O(n)$ and space complexity to $O(1)$. This chapter covers array memory layout, in-place modification, sliding windows, and prefix sums.

F.1 Array Memory Layout

An array is a contiguous block of memory. In Python, list is a dynamic array with $O(1)$ index access because the interpreter can calculate the exact memory address: $\text{address} = \text{base_address} + \text{index} * \text{element_size}$. Implications: Access by index is $O(1)$. Append to end is $O(1)$ amortized. Insert/delete at front or middle is $O(n)$ because elements must shift.

F.2 In-Place Modification Techniques

The concept: Many problems ask you to modify the array without allocating extra space ($O(1)$ extra space). This is done by overwriting the input array as you scan it. The read-write pattern: read pointer scans the entire array. write pointer tracks where the next valid element should go. When read finds something valid, it copies to write position, and write advances. Why this works: The write pointer is always less than or equal to the read pointer, so we never overwrite data we have not processed yet.

```
def remove_element(nums: list[int], val: int) -> int:
    write = 0
    for read in range(len(nums)):
        if nums[read] != val:
            nums[write] = nums[read]
            write += 1
    return write

# Time: O(n), Space: O(1)
# The write pointer never overtakes the read pointer.
```

F.3 The Sliding Window Family

The concept: A subarray (window) slides over the array. Instead of recalculating from scratch for each window, update the result by adding the new element and removing the old one. Fixed-size window: used when the window size is constant (e.g., maximum sum of k consecutive elements). Variable-size window: used when the condition is about a property, not size (e.g., longest substring without repeating characters).

```
# Fixed-size window: maximum sum of k consecutive elements
def max_sum_k(nums: list[int], k: int) -> int:
    if not nums or k > len(nums):
        return 0
    window_sum = sum(nums[:k])
    max_sum = window_sum
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]
        max_sum = max(max_sum, window_sum)
    return max_sum

# Time: O(n), Space: O(1)

# Variable-size window: longest substring without repeating characters
def length_of_longest_substring(s: str) -> int:
    char_set = set()
    left = 0
    max_len = 0
```

```

    for right in range(len(s)):
        while s[right] in char_set:
            char_set.remove(s[left])
            left += 1
        char_set.add(s[right])
        max_len = max(max_len, right - left + 1)
    return max_len

# Time: O(n), Space: O(min(m, n)) where m is charset size
# How it works:
# right expands the window, adding new characters.
# If a duplicate is found, left contracts until the duplicate is removed.
# Both pointers only move forward -> O(n) total.

```

TIP: When the problem says "in-place" or "do not allocate extra space," think two pointers immediately. The write pointer overwrites the array while the read pointer scans.

F.4 Prefix Sums & Difference Arrays

Prefix sum: An array where $\text{prefix}[i]$ is the sum of all elements from index 0 to i . Use case: Range sum queries in $O(1)$ after $O(n)$ preprocessing. Difference array: For range updates (add val to all elements from index l to r). $\text{diff}[l] += \text{val}$, $\text{diff}[r+1] -= \text{val}$. Reconstruct by taking prefix sum of diff .

```

def build_prefix(nums: list[int]) -> list[int]:
    prefix = [0] * (len(nums) + 1)
    for i in range(len(nums)):
        prefix[i + 1] = prefix[i] + nums[i]
    return prefix

# Sum of nums[i:j] = prefix[j] - prefix[i]

def range_add(diff: list[int], l: int, r: int, val: int):
    diff[l] += val
    if r + 1 < len(diff):
        diff[r + 1] -= val

```

F.5 Practice Problems

Problem 1: Move Zeroes

Given an array `nums`, move all 0s to the end while maintaining the relative order of non-zero elements. Do this in-place without making a copy of the array.

```

def move_zeroes(nums: list[int]) -> None:
    write = 0
    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write] = nums[read]
            write += 1
    for i in range(write, len(nums)):
        nums[i] = 0

# Time: O(n), Space: O(1)
# Same-direction two-pointer pattern.

```

Problem 2: Container With Most Water

Given n non-negative integers representing vertical lines where the width of each line is 1, find two lines that together with the x -axis form a container that holds the most water.

```

def max_area(height: list[int]) -> int:
    left, right = 0, len(height) - 1
    max_water = 0
    while left < right:

```



```
width = right - left
current = min(height[left], height[right]) * width
max_water = max(max_water, current)
# Move the pointer at the shorter line
if height[left] < height[right]:
    left += 1
else:
    right -= 1
return max_water

# Time: O(n), Space: O(1)
# Greedy logic: the shorter line limits the area, so moving it
# is the only way to potentially find a larger area.
```

Chapter G: Math, Logic & Number Theory

Math problems on the Odo assessment are typically straightforward: reverse digits, basic computations, prime checks, or digit manipulation. They test attention to edge cases (negative numbers, overflow, zero) rather than advanced mathematics. This chapter covers digit manipulation, primes, GCD/LCM proofs, modular arithmetic, bit manipulation, and cycle detection.

G.1 Integer Properties & Digit Manipulation

Extracting digits: Use modulo 10 to get the last digit, integer division by 10 to remove it. $n = 12345$. $\text{last_digit} = n \% 10$ gives 5. $\text{remaining} = n // 10$ gives 1234. Count digits by repeatedly dividing by 10 until zero.

```
n = 12345
last_digit = n % 10      # 5
remaining = n // 10      # 1234

# Count digits
count = 0
temp = n
while temp > 0:
    temp //= 10
    count += 1

# Alternative: count = len(str(n)) (simpler but string conversion overhead)
```

Reversing digits (mathematical approach):

Reverse the digits of a 32-bit signed integer. If reversing causes overflow (outside $[-2^{31}, 2^{31}-1]$), return 0. The key is to pop digits from the end using modulo and push them to the result using multiplication. Why `math.fmod`? Python's `%` returns a result with the same sign as the divisor (always positive for `% 10`). `math.fmod(x, 10)` preserves the sign of `x`, matching C/Java behavior.

```
import math

def reverse(x: int) -> int:
    INT_MIN, INT_MAX = -2**31, 2**31 - 1
    result = 0
    while x != 0:
        digit = int(math.fmod(x, 10))
        x = int(x / 10)
        # Overflow check before pushing
        if (result > INT_MAX // 10 or
            (result == INT_MAX // 10 and digit > 7)):
            return 0
        if (result < INT_MIN // 10 or
            (result == INT_MIN // 10 and digit < -8)):
            return 0
        result = result * 10 + digit
    return result

# Alternative for Python (simpler, but know the math version):
def reverse_simple(x: int) -> int:
    sign = -1 if x < 0 else 1
    result = sign * int(str(abs(x))[::-1])
    return result if -2**31 <= result <= 2**31 - 1 else 0
```

TIP: For interviews, know the mathematical version (modulo and division). The string conversion version is acceptable for speed in a timed test, but the interviewer may ask for the math approach in the next round.

G.2 Prime Numbers & The Sieve

Definition: A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. Naive check: Test divisibility by all numbers from 2 to $n-1$. $O(n)$. Optimized check: Test only up to $\sqrt{n}$. If n has a factor greater than $\sqrt{n}$, the complementary factor must be less than $\sqrt{n}$.

```
import math

def is_prime(n: int) -> bool:
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(math.sqrt(n)) + 1, 2):
        if n % i == 0:
            return False
    return True

# Time:  $O(\sqrt{n})$ , Space:  $O(1)$ 
# Checking only odd numbers after 2 cuts the work in half.
```

Sieve of Eratosthenes:

To find all primes up to n , iteratively mark multiples of each prime starting from 2. Why start at $i*i$? All multiples of i less than $i*i$ ($i*2, i*3, \dots, i*(i-1)$) have already been marked by smaller primes. Complexity: $O(n \log \log n)$ time, $O(n)$ space.

```
def count_primes(n: int) -> int:
    if n <= 2:
        return 0
    is_prime = [True] * n
    is_prime[0] = is_prime[1] = False
    for i in range(2, int(n**0.5) + 1):
        if is_prime[i]:
            for j in range(i * i, n, i):
                is_prime[j] = False
    return sum(is_prime)
```

G.3 GCD, LCM & The Euclidean Algorithm

GCD (Greatest Common Divisor): The largest number that divides both a and b . Euclidean Algorithm: Based on the principle that $\gcd(a, b) = \gcd(b, a \% b)$. The remainder decreases rapidly, guaranteeing termination.

Proof of correctness:

Let $d = \gcd(a, b)$. Then d divides both a and b , so d divides $a - q*b$ for any integer q . In particular, d divides $a \% b$ (since $a \% b = a - q*b$ where $q = a // b$). Thus d is a common divisor of b and $a \% b$. Conversely, any common divisor of b and $a \% b$ also divides a . Therefore the common divisors are identical, and so are the greatest common divisors.

```
def gcd(a: int, b: int) -> int:
    while b:
        a, b = b, a % b
    return a

def lcm(a: int, b: int) -> int:
    return abs(a * b) // gcd(a, b)

# Python also has math.gcd() built-in.
```

G.4 Modular Arithmetic

The concept: $(a + b) \% m = ((a \% m) + (b \% m)) \% m$. Same for subtraction and multiplication. Why useful: Prevents integer overflow and is used in hashing, cryptography, and cyclic problems.

```
# Fast modular exponentiation (a^b % mod)
def mod_pow(a: int, b: int, mod: int) -> int:
    result = 1
    a = a % mod
    while b > 0:
        if b % 2 == 1:
            result = (result * a) % mod
        a = (a * a) % mod
        b //= 2
    return result
```

G.5 Bit Manipulation Basics

Operators: & (AND) — both bits must be 1. | (OR) — at least one bit is 1. ^ (XOR) — exactly one bit is 1. ~ (NOT) — inverts all bits. << (Left shift) — multiply by 2. >> (Right shift) — integer divide by 2.

```
# Check if n is power of 2
# Binary: 8 = 1000, 7 = 0111. 8 & 7 = 0.
def is_power_of_two(n: int) -> bool:
    return n > 0 and (n & (n - 1)) == 0

# Get lowest set bit
lowest = n & (-n)

# Count set bits (Hamming weight)
def count_bits(n: int) -> int:
    count = 0
    while n:
        n &= n - 1 # clears lowest set bit
        count += 1
    return count
```

G.6 Floyd's Cycle Detection

The concept: Detect cycles in a sequence (linked list, iterated function) using two pointers moving at different speeds. Algorithm: slow moves 1 step at a time. fast moves 2 steps at a time. If there is a cycle, fast will eventually meet slow. If no cycle, fast reaches the end.

Why it works: If there is a cycle, consider the moment slow enters it. fast is already in the cycle. The distance between them decreases by 1 each step (since fast gains 1 step per iteration). Eventually the distance becomes 0.

```
def is_happy(n: int) -> bool:
    def get_next(num):
        total = 0
        while num > 0:
            digit = num % 10
            total += digit * digit
            num //= 10
        return total

    slow = n
    fast = get_next(n)

    while fast != 1 and slow != fast:
        slow = get_next(slow)
        fast = get_next(get_next(fast))

    return fast == 1
```

```
# Time:  $O(\log n)$ , Space:  $O(1)$   
# Floyd's Cycle Detection (tortoise and hare).
```

Chapter H: Python for Timed Tests

This chapter covers writing fast Python, essential standard library tools, and common pitfalls that cost candidates points in timed assessments.

H.1 Writing Fast Python

List comprehensions vs. loops: list comprehensions are faster and more pythonic. Generator expressions save memory when you only need to iterate once. String building: always use `"".join()` instead of `+=` in a loop.

```
# Fast and pythonic
squares = [x*x for x in range(1000)]

# Generator expression (no list created)
sum(x*x for x in range(1000000))

# Fast string building
result = "".join([str(x) for x in items])

# Slow (quadratic)
result = ""
for x in items:
    result += str(x)
```

H.2 Essential Standard Library Tools

collections: deque (O(1) pops from both ends), Counter (frequency counting), defaultdict (auto-creates default values). itertools: permutations, combinations, accumulate. heapq: min-heap operations. bisect: binary search in sorted lists. math: sqrt, gcd, ceil, floor, inf.

```
from collections import deque, Counter, defaultdict
from itertools import permutations, combinations, accumulate
from heapq import heappush, heappop, heapify
from bisect import bisect_left, bisect_right
from math import sqrt, gcd, ceil, floor, inf

# Counter: frequency counting
Counter([1,1,2,3,1]) # Counter({1: 3, 2: 1, 3: 1})

# defaultdict: auto-creates empty lists
graph = defaultdict(list)
graph[0].append(1) # no KeyError

# deque: O(1) pops from both ends
queue = deque([1,2,3])
queue.appendleft(0)
queue.popleft()

# heapq: min-heap
heap = [3,1,4]
heapify(heap)
heappush(heap, 2)
min_val = heappop(heap)
```

H.3 Common Python Pitfalls in Assessments

Pitfall 1: Shallow copy vs. deep copy

Using `[[0] * 3] * 3` creates three references to the same inner list. Modifying one modifies all.

```
# Wrong: all rows reference the same list
```

```
matrix = [[0] * 3] * 3
matrix[0][0] = 1 # matrix becomes [[1,0,0],[1,0,0],[1,0,0]]

# Correct: each row is independent
matrix = [[0] * 3 for _ in range(3)]
```

Pitfall 2: Mutable default arguments

Default arguments are evaluated once when the function is defined, not each time it is called.

```
# Wrong
def append(item, lst=[]):
    lst.append(item)
    return lst

# Correct
def append(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst
```

Pitfall 3: Integer division confusion

Python 3: / is float division, // is floor division (truncates toward negative infinity).

```
5 / 2 # 2.5
5 // 2 # 2
-3 // 2 # -2 (floor division!)
int(-3 / 2) # -1 (truncates toward zero)
```

Pitfall 4: Modifying a dict while iterating

This causes a RuntimeError because the dictionary size changes during iteration.

```
# Wrong: RuntimeError
for k in d:
    if condition:
        del d[k]

# Correct: iterate over a snapshot
for k in list(d.keys()):
    if condition:
        del d[k]
```

Chapter I: Mock Assessment Walkthroughs

Four complete problems solved using the R.A.P.I.D. framework, with complexity analysis and follow-up questions.

I.1 Mock Problem 1: String + Stack

Problem: Given a string containing just the characters (,), {, }, [and], determine if the input string is valid. An input string is valid if: 1. Open brackets must be closed by the same type. 2. Open brackets must be closed in the correct order.

R.A.P.I.D. Analysis:

R — Read: Input is a string of brackets. Output is boolean. A — Analyze: "()" -> True. "(){}" -> True. "()" -> False. "([])" -> False. "{}[]" -> True. P — Pattern: Nested matching -> Stack (LIFO). I — Implement: Push opening brackets. On closing bracket, check match with top of stack. D — Debug: Empty string -> True. Single bracket -> False. Very long string -> O(n) is fine.

```
def is_valid(s: str) -> bool:
    pair = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in s:
        if ch in '([{':
            stack.append(ch)
        else:
            if not stack or stack[-1] != pair[ch]:
                return False
            stack.pop()
    return len(stack) == 0

# Time: O(n), Space: O(n)
```

Follow-up (interview): What if you can only use O(1) space? Hint: You cannot use a stack for multiple bracket types, but you can use a counter for single bracket types.

I.2 Mock Problem 2: SQL Design + Query

Problem: Design a database for a simple library system with books, members, and loans. Then write a query to find all members who have currently borrowed more than 3 books.

```
CREATE TABLE members (
    member_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL
);

CREATE TABLE books (
    book_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(200) NOT NULL,
    author VARCHAR(100),
    isbn VARCHAR(13) UNIQUE
);

CREATE TABLE loans (
    loan_id INT PRIMARY KEY AUTO_INCREMENT,
    member_id INT NOT NULL,
    book_id INT NOT NULL,
    loan_date DATE NOT NULL,
    return_date DATE,
    FOREIGN KEY (member_id) REFERENCES members(member_id),
    FOREIGN KEY (book_id) REFERENCES books(book_id)
);
```



```
-- Query:
SELECT m.name, COUNT(*) AS active_loans
FROM members m
INNER JOIN loans l ON m.member_id = l.member_id
WHERE l.return_date IS NULL
GROUP BY m.member_id, m.name
HAVING COUNT(*) > 3;
```

Explanation: WHERE l.return_date IS NULL filters to currently borrowed books. GROUP BY aggregates per member. HAVING filters groups after aggregation. We include m.name in GROUP BY because some SQL modes require all selected non-aggregated columns to be in GROUP BY.

I.3 Mock Problem 3: Graph BFS

Problem: You are given an undirected graph with n nodes, labeled 0 to $n-1$, and a list of edges. Find the shortest path length from node 0 to node $n-1$. If no path exists, return -1.

R.A.P.I.D. Analysis:

R — Read: Graph, unweighted, undirected. Find shortest path length. A — Analyze: $n=4$, edges=[[0,1],[1,2],[0,2],[2,3]]. Path 0->2->3 has length 2. P — Pattern: Shortest path in unweighted graph -> BFS. I — Implement: Build adjacency list, BFS from 0, return distance when reaching $n-1$. D — Debug: $n=1$ (start=end) -> 0. No edges -> -1. Disconnected graph -> -1.

```
from collections import deque

def shortest_path(n: int, edges: list[list[int]]) -> int:
    if n == 1:
        return 0
    graph = {i: [] for i in range(n)}
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)
    visited = set([0])
    queue = deque([(0, 0)])
    while queue:
        node, dist = queue.popleft()
        for neighbor in graph[node]:
            if neighbor == n - 1:
                return dist + 1
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, dist + 1))
    return -1

# Time: O(V + E), Space: O(V)
# Why not DFS? DFS might find a longer path first and has no
# mechanism to know if a shorter one exists.
```

I.4 Mock Problem 4: Array Two-Pointer

Problem: Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target. You may assume each input has exactly one solution, and you may not use the same element twice.

R.A.P.I.D. Analysis:

R — Read: Array, target. Return indices. A — Analyze: nums=[2,7,11,15], target=9 -> [0,1]. P — Pattern: Finding pairs with a sum -> Hash set for $O(n)$, or two-pointer if sorted (but returning indices complicates sorting). I — Implement: Hash map storing value -> index. For each num, check if complement exists in map. D — Debug: Two identical numbers that sum to target (e.g., [3,3], target=6) -> map handles this because we check before inserting.

```
def two_sum(nums: list[int], target: int) -> list[int]:
```

```
seen = {} # value -> index
for i, num in enumerate(nums):
    complement = target - num
    if complement in seen:
        return [seen[complement], i]
    seen[num] = i
return [] # should never reach here per problem statement

# Time: O(n), Space: O(n)
# Why not two-pointer here? If we sort, we lose original indices.
# We would need to track indices with tuples, making the code more complex.
```

Appendix: The 60-Minute Cheat Sheet

Print this page or keep it beside your screen for rapid lookup during practice.

Python String Methods

`s.strip()`, `s.split()`, `s.join()`, `s.replace()`, `s.find()`, `s.count()`, `s.startswith()`, `s.endswith()`, `s.isdigit()`, `s.isalpha()`, `s.isalnum()`, `s[::-1]`

Collections

from collections import deque, Counter, defaultdict. deque: `appendleft()`, `pop()`, `popleft()`. Counter: `most_common()`, `elements()`. `defaultdict(list)`: auto-creates empty lists.

SQL Quick Patterns

SELECT ... FROM ... JOIN ... ON ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT. INNER JOIN: matching rows only. LEFT JOIN: all left rows, NULL for missing right. RANK() OVER (PARTITION BY ... ORDER BY ...). Aggregate: COUNT, SUM, AVG, MAX, MIN.

Graph Templates

Adjacency list: `graph = {}`; `graph.setdefault(u, []).append(v)`. BFS: `queue = deque([(start, 0)])`; while `queue`: `node, dist = queue.popleft()`. DFS recursive: `def dfs(node): visited.add(node); for nbr in graph[node]: dfs(nbr)`. DFS iterative: `stack = [start]`; while `stack`: `node = stack.pop()`.

Stack & Queue

Stack (LIFO): `list.append()`, `list.pop()`, `list[-1]`. Queue (FIFO): `deque.append()`, `deque.popleft()`. Monotonic stack: while `stack` and condition: `stack.pop()`.

Two Pointers

Opposite: `left, right = 0, len(arr)-1`; while `left < right`. Same-direction: `write = 0`; for `read` in `range(len(arr))`: ... Sliding window: `window_sum += new - old`; `max_sum = max(max_sum, window_sum)`.

Math Snippets

Reverse digits: `result = result * 10 + digit`. Prime check: `range(2, int(sqrt(n))+1)`. GCD: while `b`: `a, b = b, a % b`. Power of two: `n > 0 and (n & (n-1)) == 0`. Modular arithmetic: `(a + b) % m == ((a % m) + (b % m)) % m`.

Big-O Reminders

Nested loops over `n`: usually $O(n^2)$. Single pass with hash set: $O(n)$ time, $O(n)$ space. Two pointers on sorted array: $O(n)$ time, $O(1)$ space. BFS/DFS on graph: $O(V + E)$. Binary search: $O(\log n)$.

Final Advice:

1. Start with the easiest problem. Build momentum.
2. Write brute force if stuck. A working slow solution is better than a broken fast one.
3. Explain your code to yourself. If you cannot explain it, the interviewer will catch you.
4. Test edge cases mentally. Empty input, single element, all same elements, maximum values.
5. Keep calm. You have prepared. Trust the process.

Good luck.